



Artificial Intelligence Fundamentals

DESIGN OF INTELLIGENT AGENTS

Agents and their environment

Agent is anything that can act (duh)

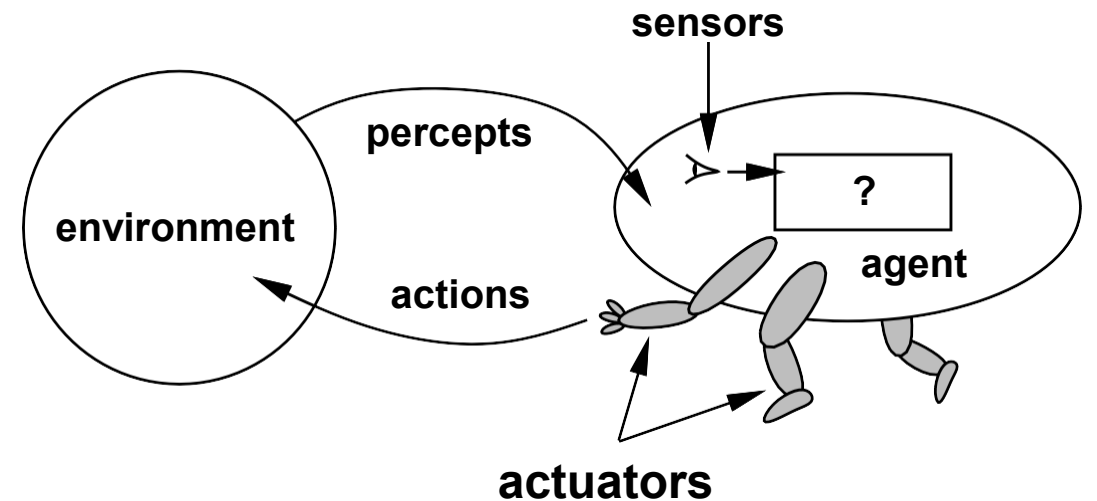
Equipped with:

- Sensors – to perceive the environment
- Actuators – to intervene in the environment

Agent function (or policy) $f: P^* \rightarrow A$

- P^* is the history of all percepts
- A is the set of possible agent's actions

f can be physically implemented or simulated



Vacuum-cleaner world

Agent: vacuum-cleaner

$A = \{Left, Right, Suck, NoOp\}$

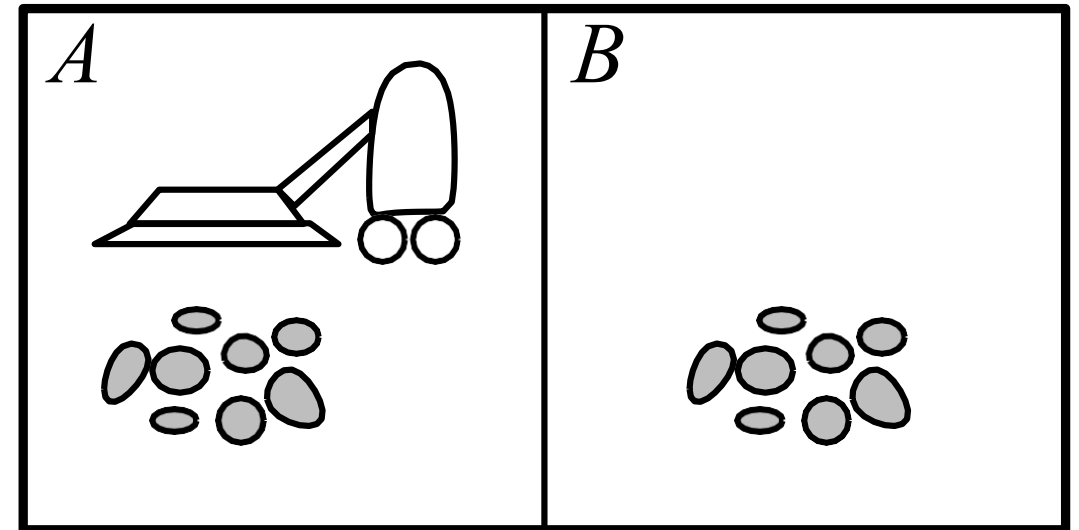
Agent position $x \in \{A, B\}$

Environment state S

- Accessible only through perception (observed environment state)
- $s_t \in S = \{clean, dirty\}$

Percepts: $\{(x_t, s_t)\}_{t=1, \dots}$

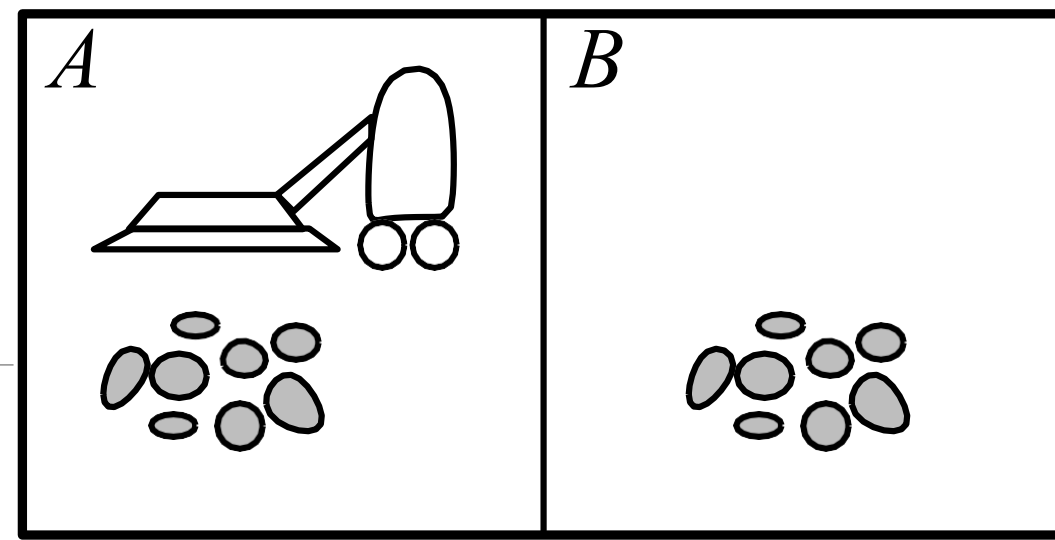
Objective (the task): to clean the environment



Your first agent

What do we need to do?

- Build f



Percept sequence	Action
$[A, \text{Clean}]$	<i>Right</i>
$[A, \text{Dirty}]$	<i>Suck</i>
$[B, \text{Clean}]$	<i>Left</i>
$[B, \text{Dirty}]$	<i>Suck</i>
$[A, \text{Clean}], [A, \text{Clean}]$	<i>Right</i>
$[A, \text{Clean}], [A, \text{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$
$[A, \text{Clean}], [A, \text{Clean}], [A, \text{Clean}]$	<i>Right</i>
$[A, \text{Clean}], [A, \text{Clean}], [A, \text{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$

Your first agent

Let's «code»!

Percept sequence	Action
<i>[A, Clean]</i>	<i>Right</i>
<i>[A, Dirty]</i>	<i>Suck</i>
<i>[B, Clean]</i>	<i>Left</i>
<i>[B, Dirty]</i>	<i>Suck</i>
<i>[A, Clean], [A, Clean]</i>	<i>Right</i>
<i>[A, Clean], [A, Dirty]</i>	<i>Suck</i>
<i>⋮</i>	<i>⋮</i>
<i>[A, Clean], [A, Clean], [A, Clean]</i>	<i>Right</i>
<i>[A, Clean], [A, Clean], [A, Dirty]</i>	<i>Suck</i>
<i>⋮</i>	<i>⋮</i>

```
function Reflex-Vacuum-Agent( [location,status]) returns an action
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

Does this make sense?

Intuitively, yes.

Rational agent: the agent that does the «right thing»

- For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has

Consequentialism: evaluate an agent based on its actions' consequences

Looks intuitive, but it comes with a whole lot of work to do

Performance metric

Goodhart's law: «when a measure becomes a target, it ceases to be a good measure»

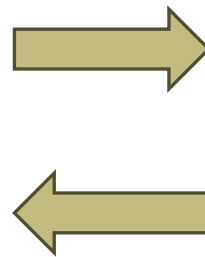
Performance metric evaluates the “goodness” of an environment state / state sequence.

How to evaluate a vacuum-cleaner agent?

Reasonable assumption: no access to the room to check if it is clean

Performance metric: grams of dirt picked up over a 1-hour shift.

We can now build agents that maximize this metric. What could go wrong?



The rational dilemma

A rational agent chooses its actions in order to maximize the performance metric, given the percept history

- **Rational $\neq$ omniscient**, percepts may not supply all relevant information
- **Rational $\neq$ clairvoyant**, we don't have access to future percepts
- **Rational $\neq$ successful**, action outcomes may not be as expected

In any case, careful engineer of performance metric and control of the learned behavior are key.

Static environments

Static environments that are completely **observable** and **predictable** encourage to learn a fixed behavior.

The optimal solution does not change.

This leads to **fragile** agents following [Fixed Action Patterns](#)

- An agent continues to follow its policy
- Sphex wasp: after digging a burrow, it goes out and sting a caterpillar and drag it to the burrow, enter the burrow again to check all is well, drag the caterpillar inside, and lay its eggs.. If an entomologist moves the caterpillar a few inches away while the Sphex is doing the check, it will revert to the “drag the caterpillar” step of its plan and will continue the plan without modification, re-checking the burrow, even after dozens of caterpillar-moving interventions.



PEAS – describe a task for a rational agent

Performance metric

Environment

Actuators

Sensors

Agent architecture = A + S

Agent = architecture + function

PEAS – autonomous taxi service

Performance metric: *safety, destination, profits, legality, comfort, . . .*

Environment: *streets/freeways, traffic, pedestrians, weather, . . .*

Actuators: *steering, accelerator, brake, horn, speaker/display, ...*

Sensors: *camera, accelerometers, gauges, engine sensors, keyboard, GPS...*

The more restricted the environment, the easier the agent's design.

Different types of environments

Fully Observable, Partially
Observable, Unobservable

Deterministic, Stochastic/Non-
deterministic

Discrete, Continuous

Single Agent, Multi Agent

Observable?

Fully observable

sensors give access
to the *complete* state
of the environment

Partially observable

faulty sensors,
missing information

Unobservable

no sensor info at all

Deterministic?

Yes $\rightarrow$ (state, action) **uniquely** determines the next state

No $\rightarrow$ the same (state, action) can lead to different next states

- Partial observability can be one cause of non-determinism

Discrete?

State of the environment

Time

Percepts

Actions

They can be represented by **discrete-** or **continuous-**values

Single agent?

Does the performance of my agent depend on other agent's actions?

Bonus: known vs. unknown

In known environment, the agent knows the *rules* of the environment

- Can still be partially observable

Otherwise, the agent might try to learn the rules

- (state, action) → next state

Different environments

Characteristic	Sudoku	Backgammon	Taxi service
Fully Observable	Yes	Yes	No
Deterministic	Yes	No	No
Discrete	Yes	Yes	No
Single-agent	Yes	No	No

Which agent to build? Define the environment, first!

What about the real world?

Different types of agent's functions: table-driven agent

Only feasible in simple
environments

Complexity of the environment +
agent will make the table size too
large

Effectiveness-efficiency trade-off

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
               table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

Different types of agent's functions: simple reflex agent

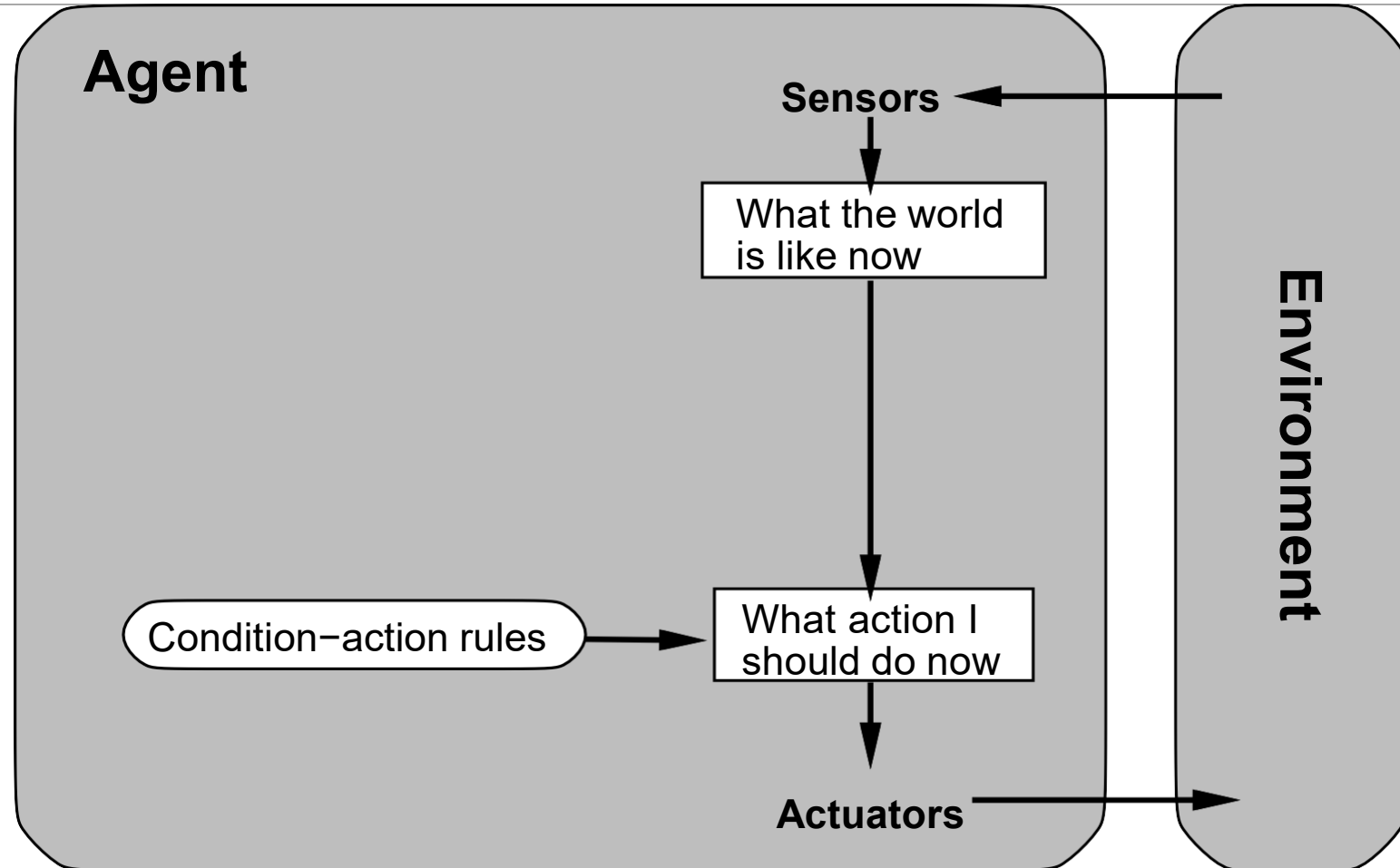


Table agent vs. simple reflex agent

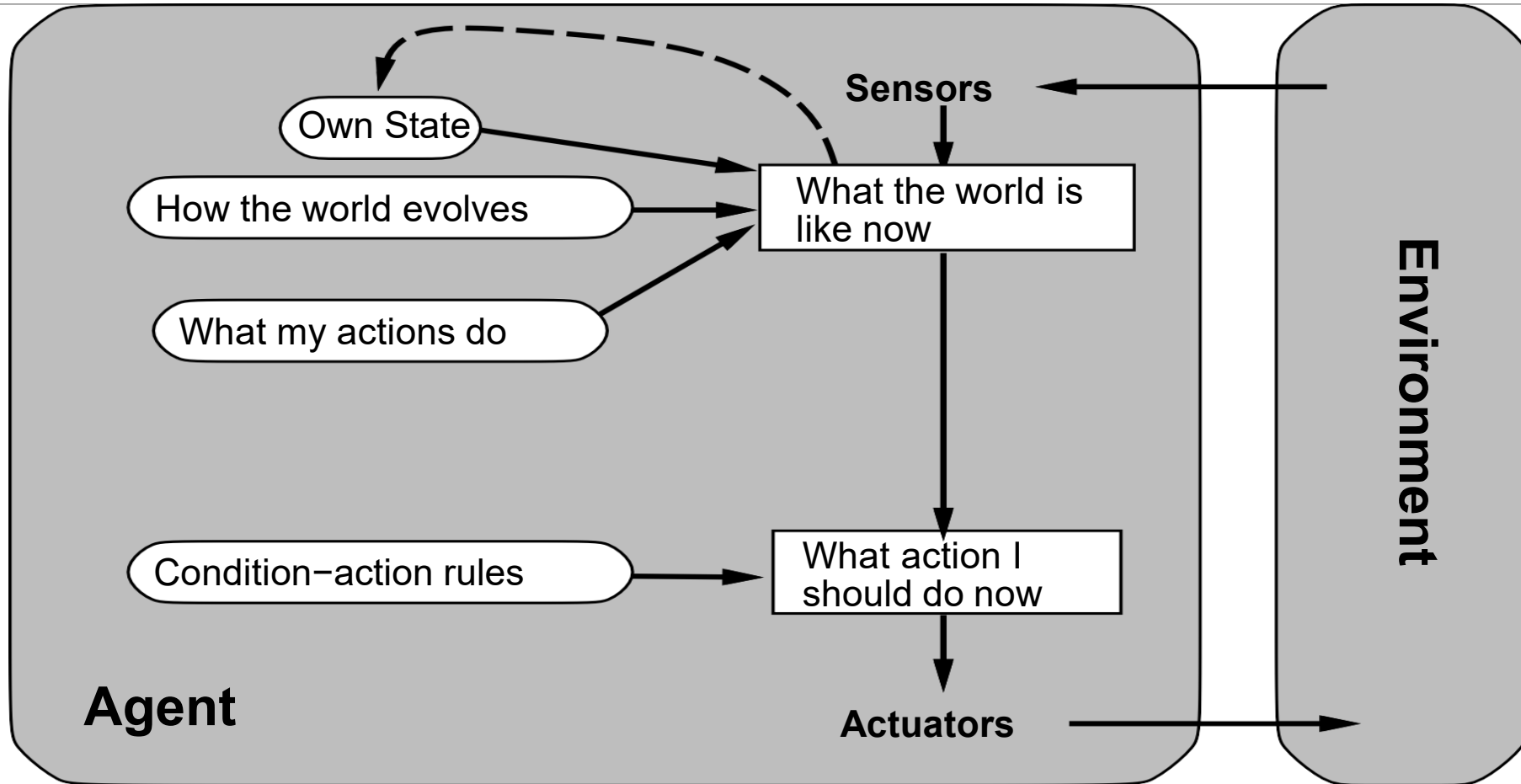
SRA dramatically cuts down costs

- TA scales exponentially with time, SRA scales linearly with time

SRA further simplifies an already naïve approach

- Only works in simple environments (small, fully-observable environments are rare).

Different types of agent's functions: model-based agent



Model-based agent: transition + sensor

Estimate the current environment state based on the agent's **internal state**

- The internal state depends on percept history

The internal state is modeled by

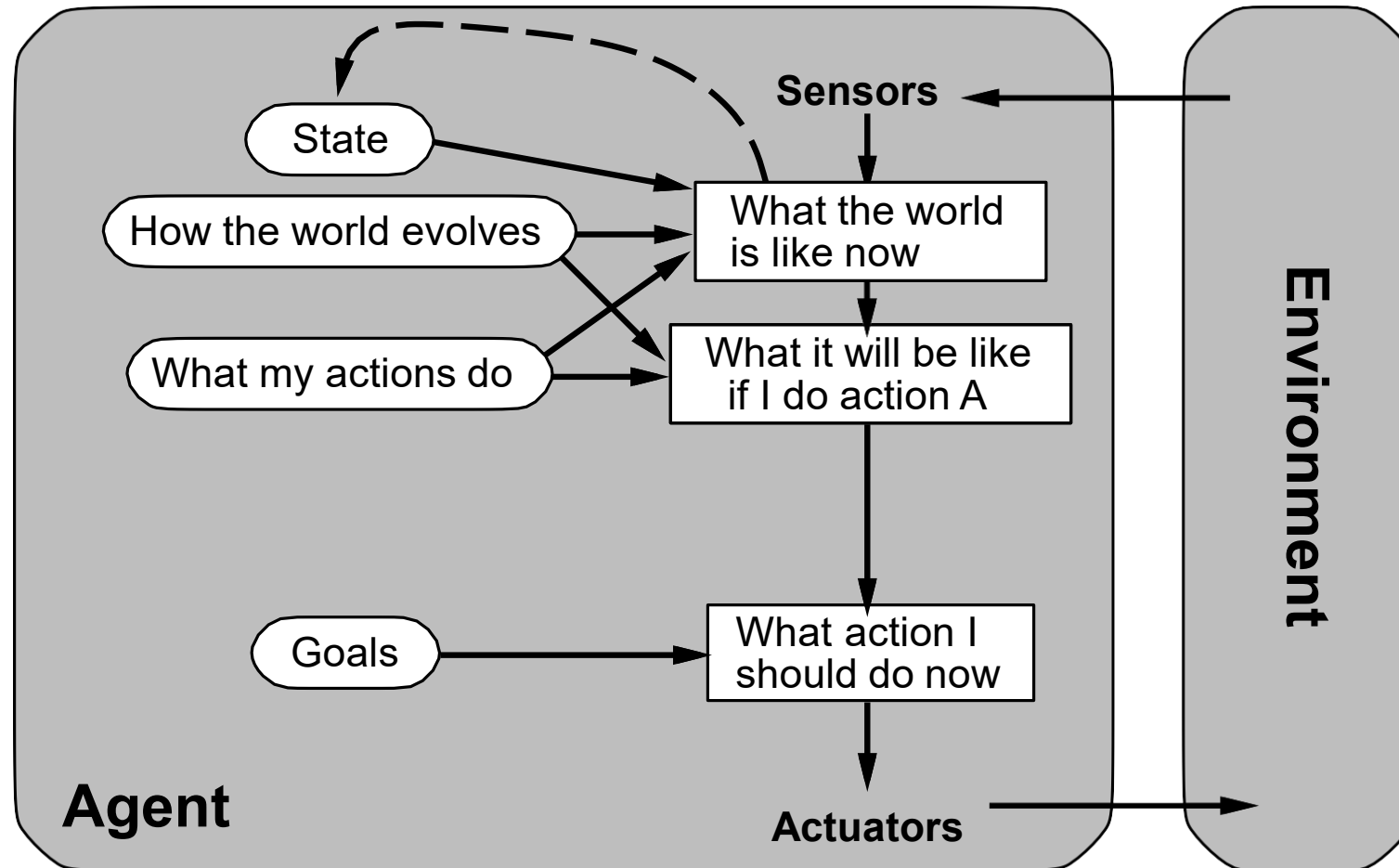
- The **transition model** (consequences of agent's actions)
- The **sensor model** (how the agent's sensor represent the world)

Effective in handling *partially observable environments* by approximating the entire state

State-of-the-art: LLMs are struggling to acquire a model of the world

[LLMs and World Models, Part 1 - by Melanie Mitchell](#)

Different types of agent's functions: goal-based agent



How to choose the best action?

Maximize the immediate reward vs. maximize the expected long-term reward

Search: how to best navigate the possible options (decision tree)

Planning: leverage the world's knowledge to execute the best actions

Compared w. reflex-agent:

- What happens if you decide to change goal?

The reflex agent has no notion of «goal», it needs to be rebuilt from scratch as its condition-action rules only model one problem.

The goal-based agent represents its own goals and **can** change them.

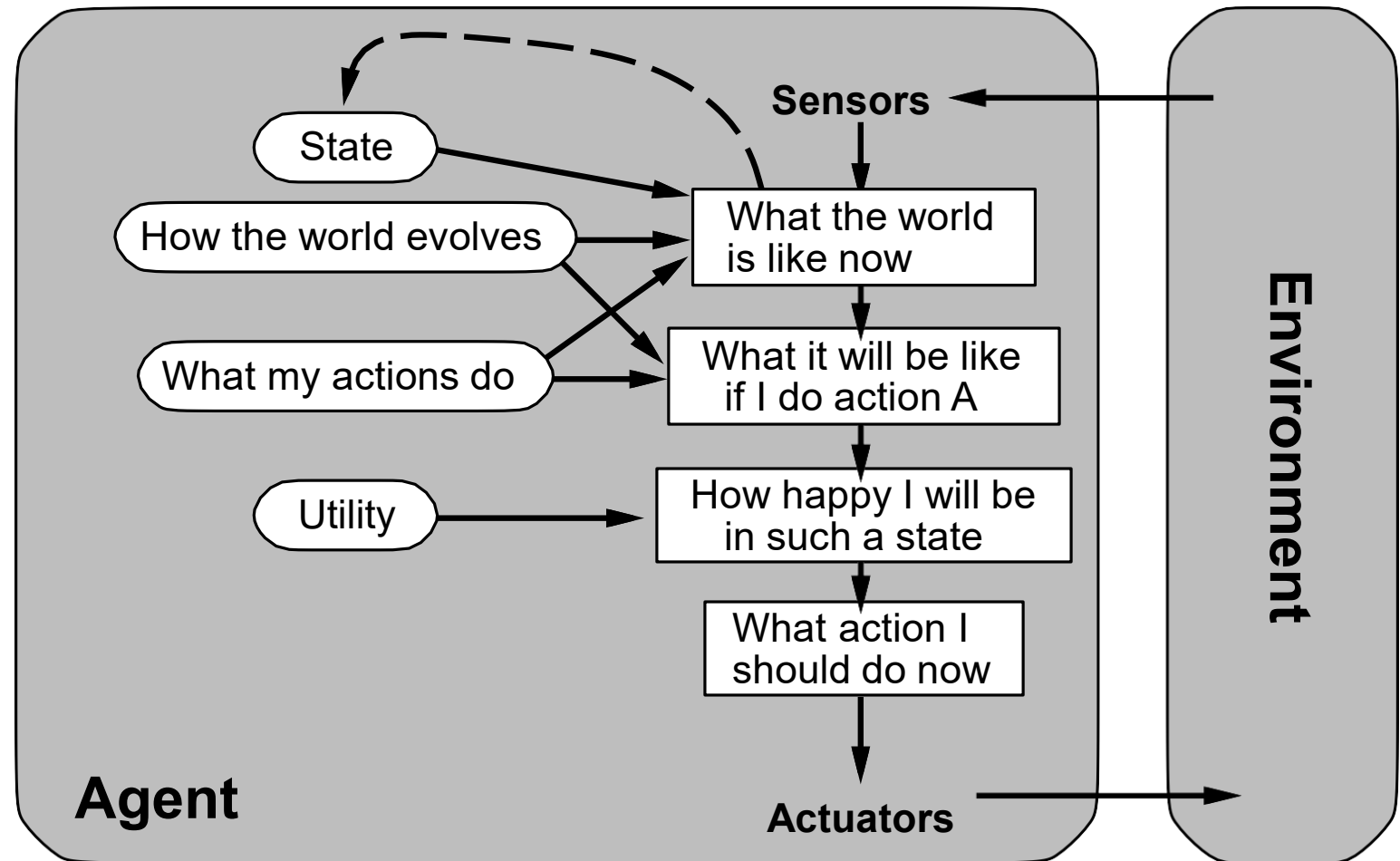
Different types of agent's functions: utility-based agent

Internal notion of **utility** to assign a value to each *intermediate* environment state (or sequence of states) instead of only to the final goal

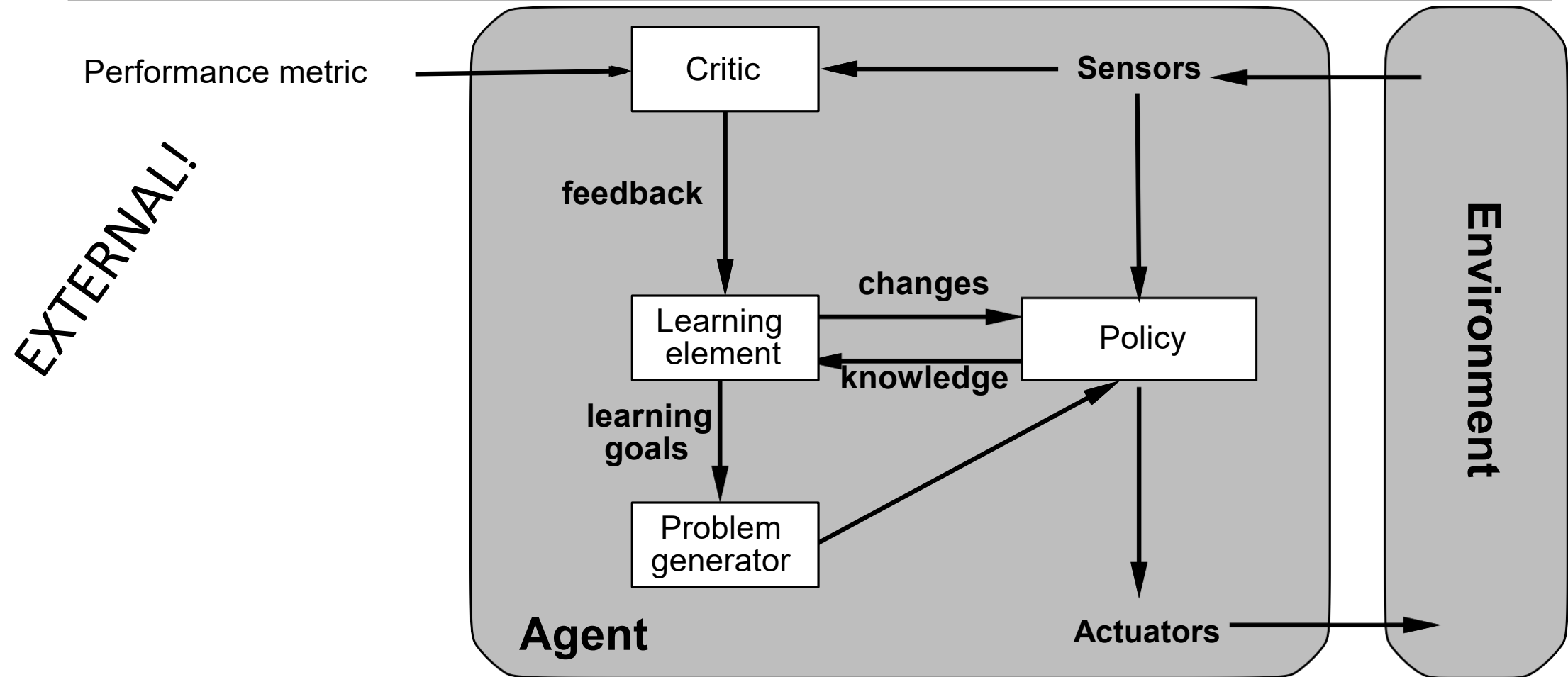
Like an internal performance measure

Often used in stochastic, partially observable environments to maximize the *expected* utility for each possible action's result i :

$$\text{Expected utility} = \sum_i p_i u_i$$



Different types of agent's functions: learning-based agent



Learning vs. all?

Any previously mentioned agent can be a learning-based agent or not.
Learning can modify any component.

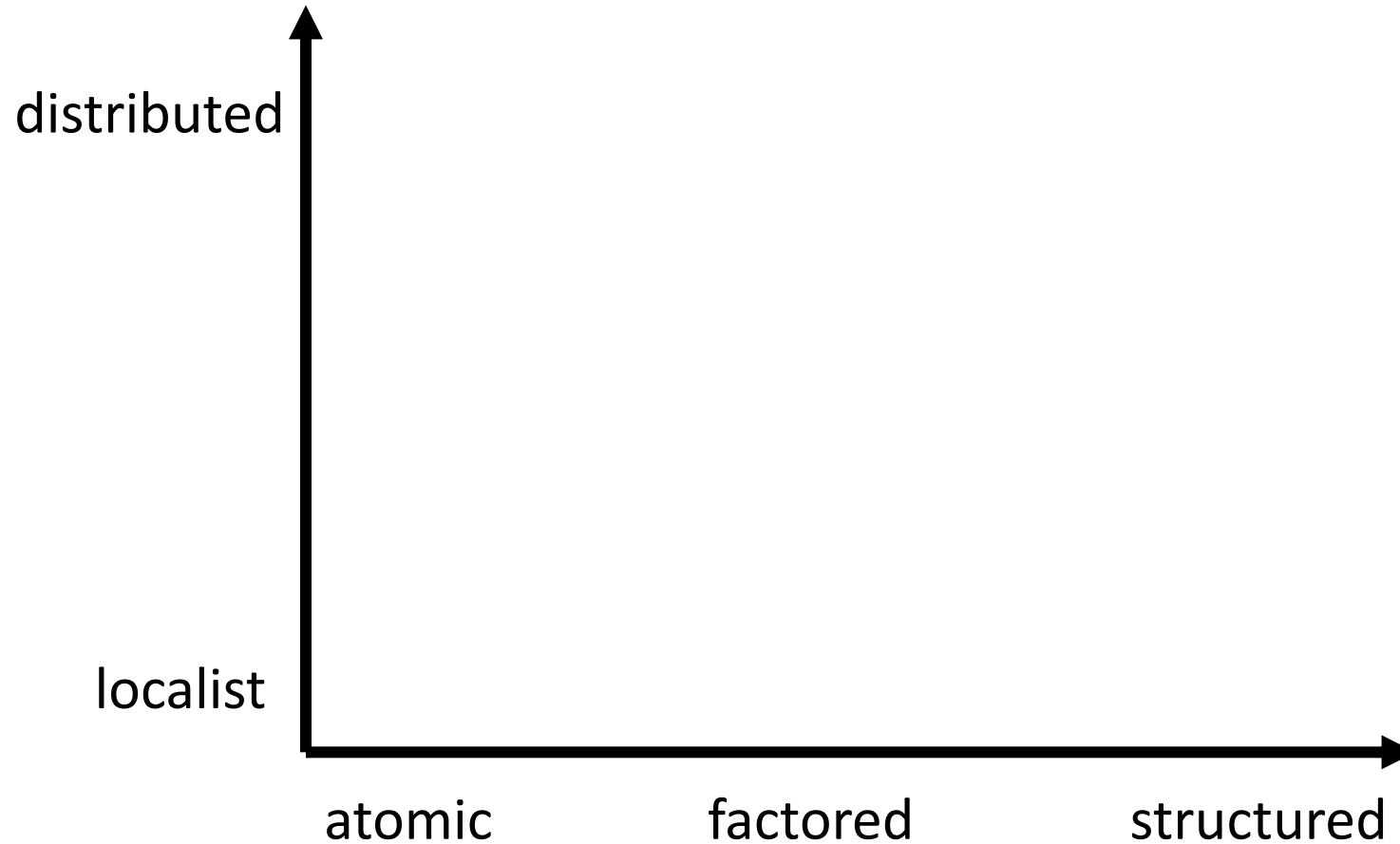
Policy: given the context (percepts + internal state), choose an action

Critic: drive the learning by evaluating the current behavior wrt **external** performance metric

Problem generator: steer away from greedy behavior (balance the critic), try out new actions

to suggest these exploratory actions. This is what scientists do when they carry out experiments. Galileo did not think that dropping rocks from the top of a tower in Pisa was valuable in itself. He was not trying to break the rocks or to modify the brains of unfortunate pedestrians. His aim was to modify his own brain by identifying a better theory of the motion of objects.

Knowledge representation in agents:



Knowledge representation in agents

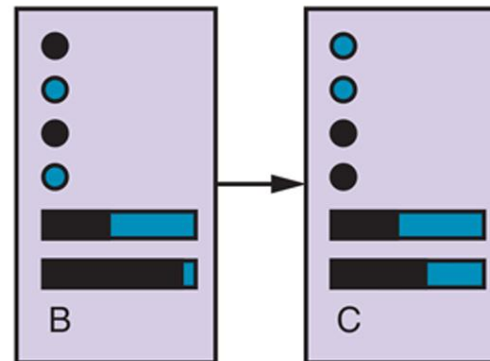
Atomic

- no internal state structure



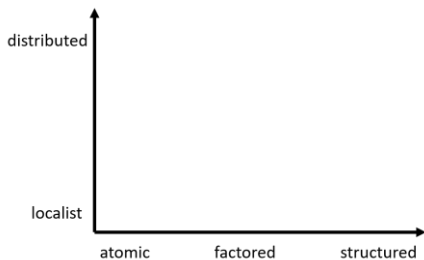
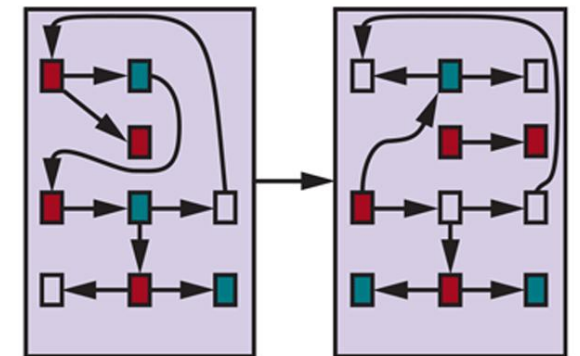
Factored

- Vector of state values (attributes)



Structured

- Graph structure: objects w. properties and relationships

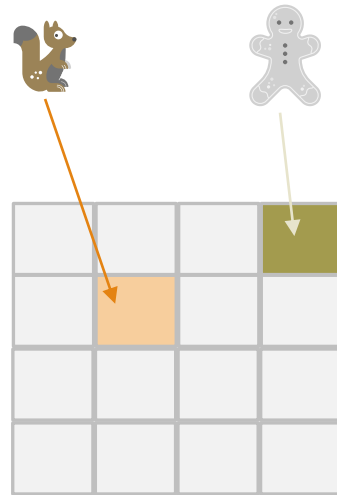


Expressiveness

Knowledge representation in agents

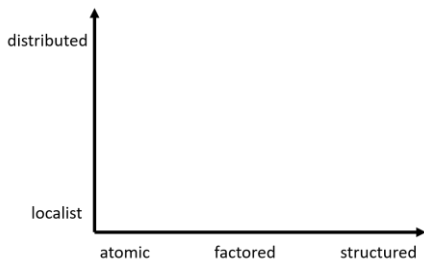
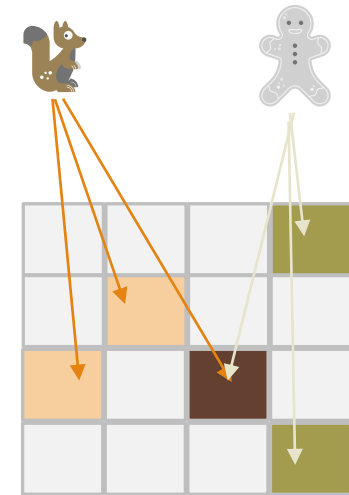
Localist

- One-to-one mapping between concepts and memory locations



Distributed

- A memory cell is used in multiple concepts
- A concept is represented by multiple cells



Summary

Agents interact with **environments** through **actuators** and **sensors**

The agent **function** describes what the agent does in all circumstances

The **performance measure** evaluates the environment sequence

A perfectly rational agent maximizes expected performance

PEAS descriptions define task environments

Environments are categorized along several dimensions:

- observable? deterministic? episodic? static? discrete? single-agent?

Several types of agents exist:

- reflex, reflex with state, goal-based, utility-based, ...

Agent's knowledge can be more or less distributed / expressive

Upcoming – Projects & Search

